

MODULE 5

The Aggregation Framework

Turn training_store documents into totals, rankings and reports — one pipeline stage at a time.

Pipelines

Filter and shape

Group

Arrays and joins

Reports

LEARNING OBJECTIVES

What you will be able to do by the end of this module.

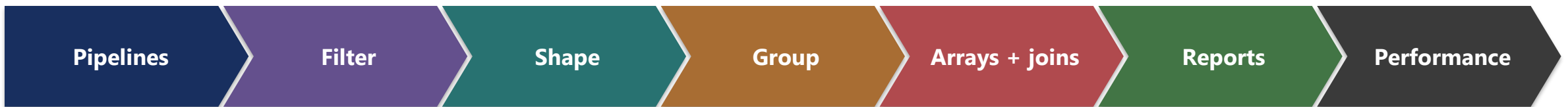


BY THE END OF THIS MODULE, YOU CAN

1. Explain how documents flow through a pipeline
2. Filter and reshape with \$match, \$project, \$set
3. Summarize with \$group and accumulators
4. Expand items with \$unwind; join with \$lookup
5. Build one report with several metrics
6. Order stages for correct, cheap pipelines



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHAT

is an aggregation pipeline?

HOW

do we filter and shape documents?

HOW

do we summarize many documents?

HOW

do we use arrays and other collections?

HOW

do we build one multi-metric report?

WHY

does stage order matter?

KEY TAKEAWAY A pipeline answers questions no single stored document can — one stage at a time.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 Aggregation pipeline

An ordered array of stages that `db.collection.aggregate()` runs on a stream of documents.

2 Stage

One step such as `$match`, `$group` or `$sort`: it takes documents in and passes new documents on.

3 Field path

`"$total"` or `"$items.quantity"`: the `$` tells MongoDB to read a field's value, not the text.

4 Expression

A calculation evaluated per document, such as `$multiply`, `$concat` or `$cond`.

5 Grouping key

The `_id` of `$group`: every distinct value becomes one output document.

6 Accumulator

An operator that combines values across a group: `$sum`, `$avg`, `$min`, `$max`, `$push`.

7 \$unwind

Turns one document with an array into one document per array element.

8 \$lookup

A left outer join to another collection; the matches arrive as an array.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

API

Application Programming Interface

CRUD

Create, Read, Update, Delete

SKU

Stock Keeping Unit

BSON

Binary JSON

ID

Identifier

UTC

Coordinated Universal Time

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 4 TO MODULE 5

find() reads what is stored — aggregate() computes what isn't.

MODULE 4 GAVE US

- ✓ find() filters with query operators
- ✓ Projections pick the returned fields
- ✓ Dotted paths reach nested fields
- ✓ sort() and limit() rank results
- ✓ Updates change stored documents

NOW Answer questions find() can't: totals, rankings and joins.



OUR PATH THROUGH MODULE 5

- 1 How a pipeline works; why order matters
- 2 Filter, reshape and compute fields
- 3 Group and summarize with accumulators
- 4 Expand items; join customers
- 5 Dates, price bands and one report
- 6 Stage order, cost and debugging

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

KEY TAKEAWAY Module 4's filters become the first stage of every pipeline in Module 5.

THE FIELDS EVERY PIPELINE USES

The real paths in training_store — and the traps the data is built to teach.



TRAINING_STORE AFTER A FRESH LOAD

customers

6 documents

products

13 documents

orders

17 documents

reviews

6 documents



ONE ORDER: THE PATHS WE AGGREGATE

```
{ orderNumber: "06301",
  customerId: ObjectId("..."),    // → customers._id
  items: [ { sku: "L100", name: "Business Laptop",
             quantity: 1,
             unitPrice: Decimal128("1299.99") }, ... ],
  paymentStatus: "PAID",
  fulfillmentStatus: "SHIPPED",
  subtotal: ..., tax: ..., shippingFee: Decimal128("15.00"),
  total: Decimal128("1512.23"),
  createdAt: ISODate("2026-09-03T10:15:00Z") }
```

TRAPS BUILT INTO THE DATA

- ! No status field: use paymentStatus and fulfillmentStatus
- ! No lineTotal: compute quantity × unitPrice
- ! shippingFee is missing on 6 orders
- ! XBAD's price is the string "49.99"
- ! O6499's customerId matches no customer

MONEY Decimal128 — compare with
Decimal128("1000.00").

Before Lab 4

Count each collection and write down these paths before any pipeline.

KEY TAKEAWAY A pipeline is only as correct as its field paths — check them against a real document first.

PART 1: HOW A PIPELINE WORKS

Documents in, stages in order, new documents out.



SO FAR

- ✓ You know `find()`, projections and `sort()`
- ✓ You know the `training_store` field paths

NEXT QUESTION

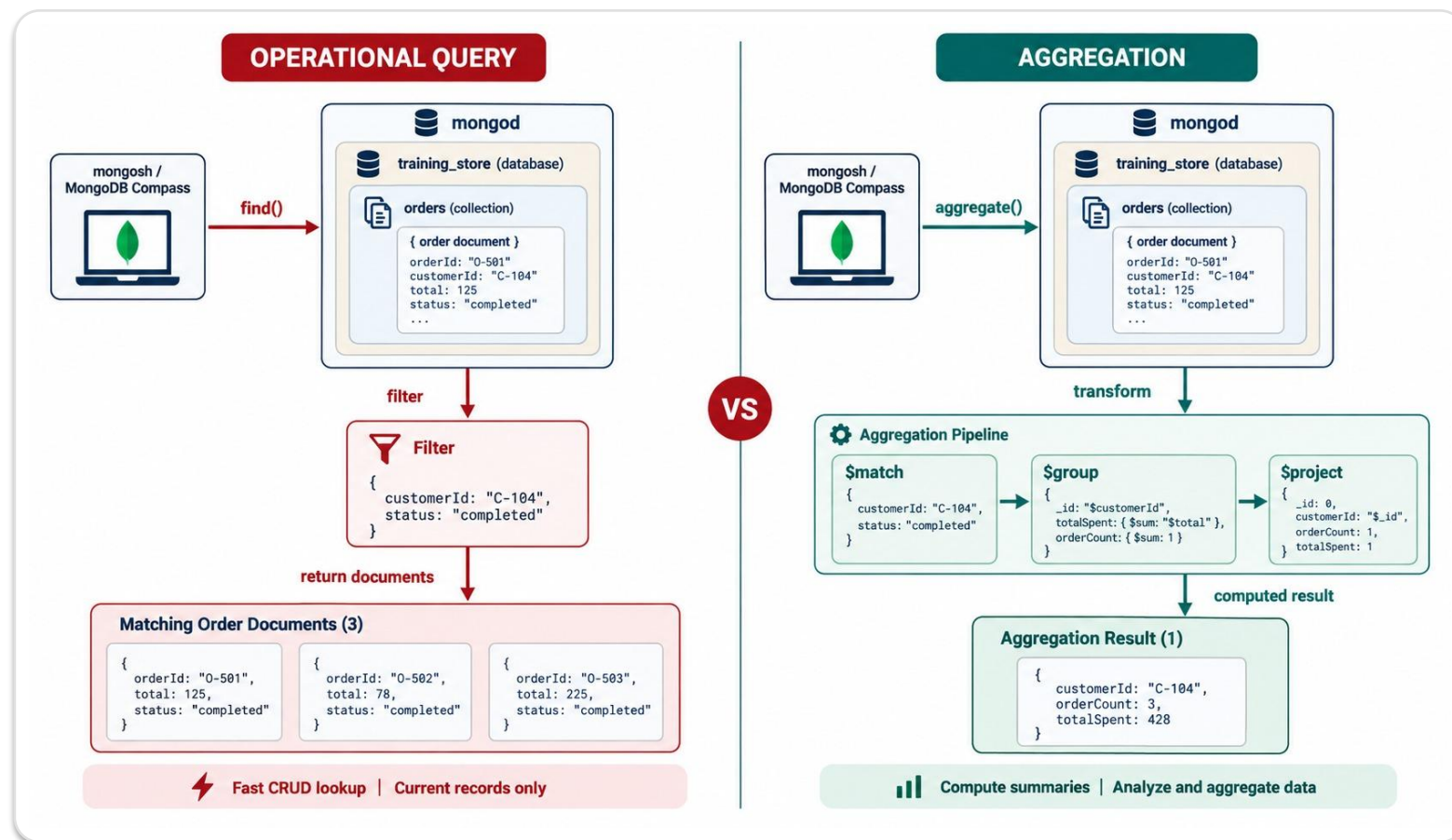
What does a pipeline do that `find()` can't — and why does the order of its stages matter?

IN THIS PART

- 1 Queries versus aggregation
- 2 How documents flow through stages
- 3 One pipeline, stage by stage
- 4 Why stage order matters

OPERATIONAL QUERIES VS. AGGREGATION

find() returns stored documents; *aggregate()* computes new ones.



find()

Returns the stored documents that match — everyday CRUD reads.

aggregate()

Computes new documents: counts, totals, averages, rankings, joins.

Our example

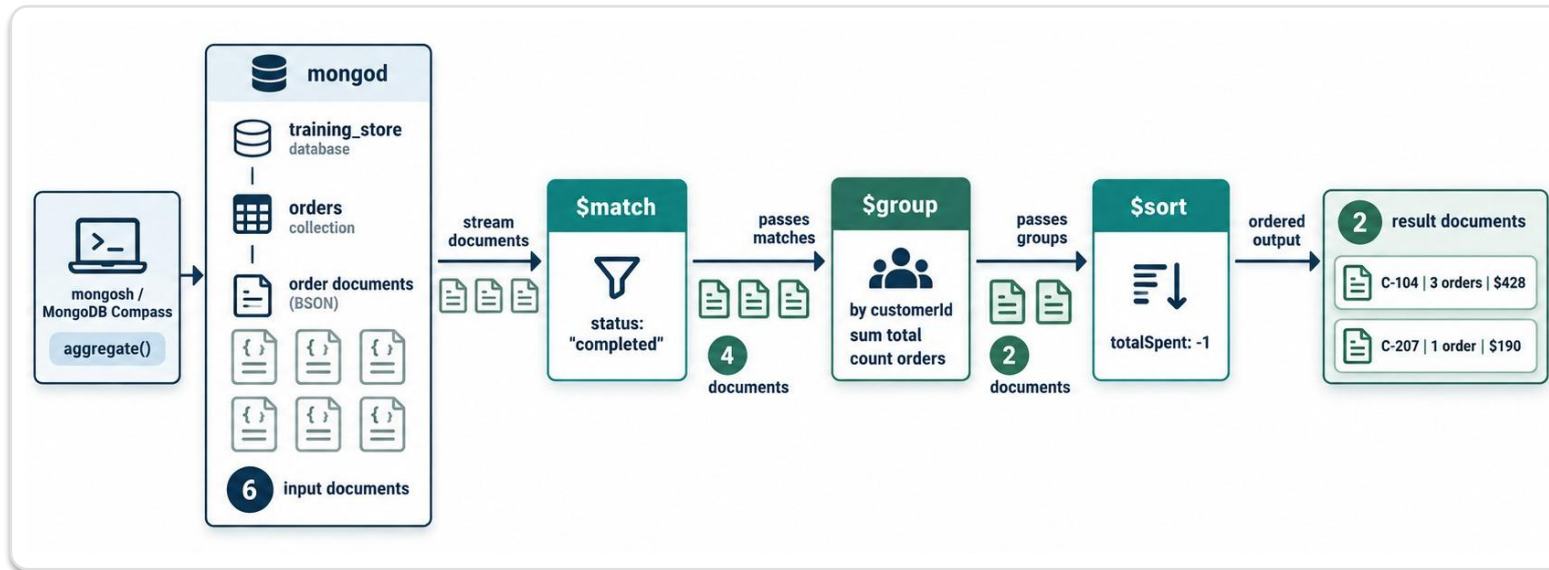
find() lists Aisha's paid orders; aggregate() says they total 3418.62.

RULE OF THUMB Not stored in one document? Aggregate.

KEY TAKEAWAY Use find() to retrieve documents and aggregate() to compute answers from them.

HOW DOCUMENTS MOVE THROUGH A PIPELINE

An array of stages; each stage's output is the next stage's input.



An array of stage documents

```
db.orders.aggregate([
  { $match: { ... } },
  { $group: { ... } },
  { $sort: { ... } }
])
```

A stream

Stages run in array order; each sees only the last one's output.

Count and shape change

6 → 4 → 2: **\$match** shrinks the stream, **\$group** reshapes it.

READ-ONLY Stored orders never change.

KEY TAKEAWAY A pipeline is a stream: each stage transforms the documents and passes them on.

DEMO: ONE PIPELINE, STAGE BY STAGE

Paid revenue by fulfillment status — run it one stage at a time.



THE PIPELINE

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $group: {
    _id: "$fulfillmentStatus",
    orders: { $sum: 1 },
    revenue: { $sum: "$total" } } },
  { $sort: { revenue: -1 } }
])
```

THE RESULT: 4 DOCUMENTS

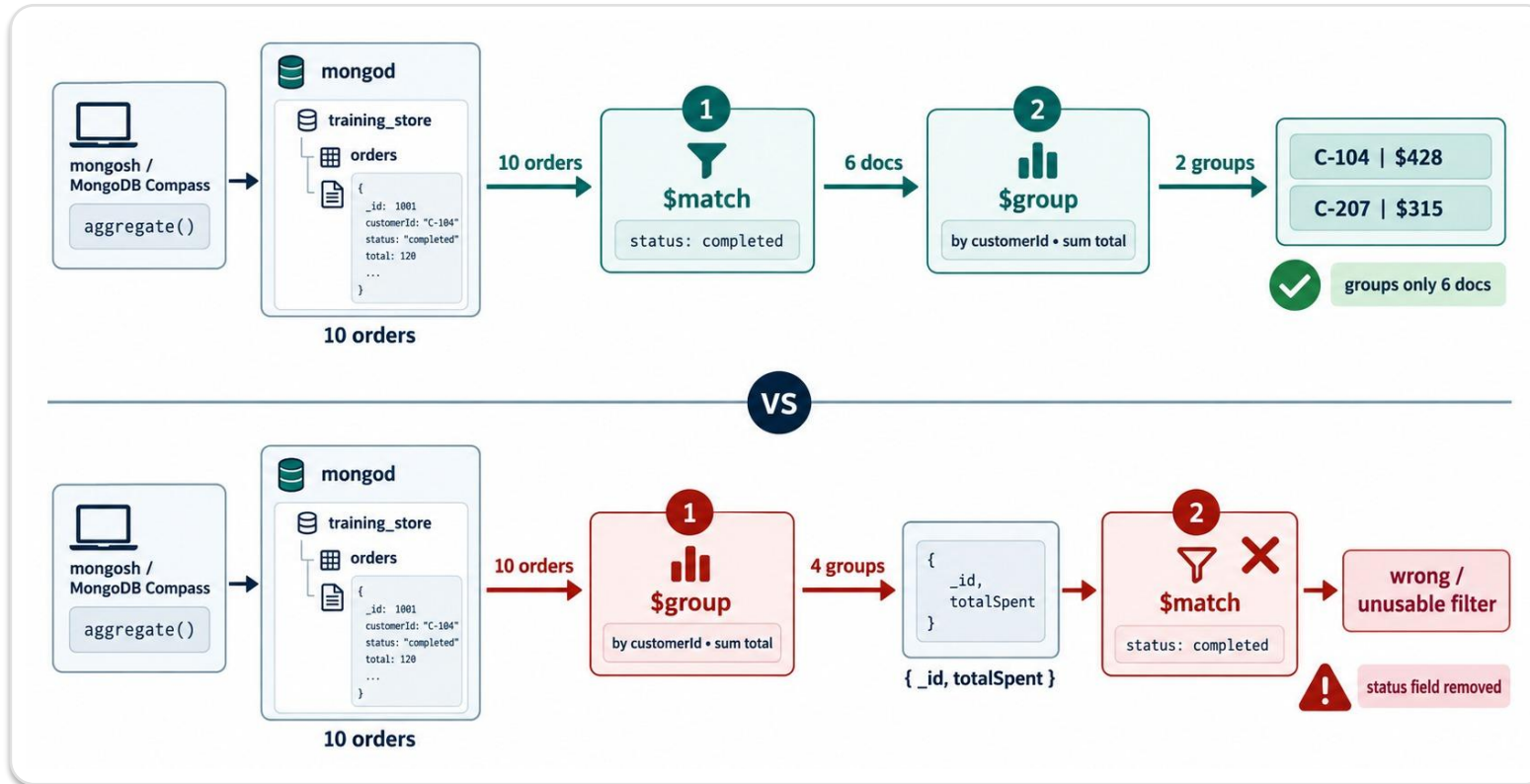
_id	orders	revenue
SHIPPED	6	3809.48
DELIVERED	4	2788.42
PROCESSING	2	2448.77
NEW	1	71.77

Check: $6 + 4 + 2 + 1 = 13$ paid orders.

KEY TAKEAWAY Run each stage on its own first — the counts tell you whether it did what you meant.

STAGE ORDER MATTERS

Match then group is a report; group then match returns nothing.



Match, then group

Keep PAID orders, then total them: paid revenue per customer.

Group, then match

`$group` keeps only `_id` and totals — `paymentStatus` is gone.

Wrong order on training_store

```
{ $group: { _id: "$customerId",  
  spent: { $sum: "$total" } } },  
{ $match: { paymentStatus: "PAID" } }  
// → 0 documents
```

KEY TAKEAWAY Stage order is part of the question — filter on stored fields before `$group` removes them.

PART 2: FILTER AND SHAPE

Choose the documents, then choose — or compute — their fields.



SO FAR

- ✓ A pipeline is an ordered array of stages
- ✓ Each stage sees only the previous output
- ✓ Filter before \$group removes fields

NEXT QUESTION

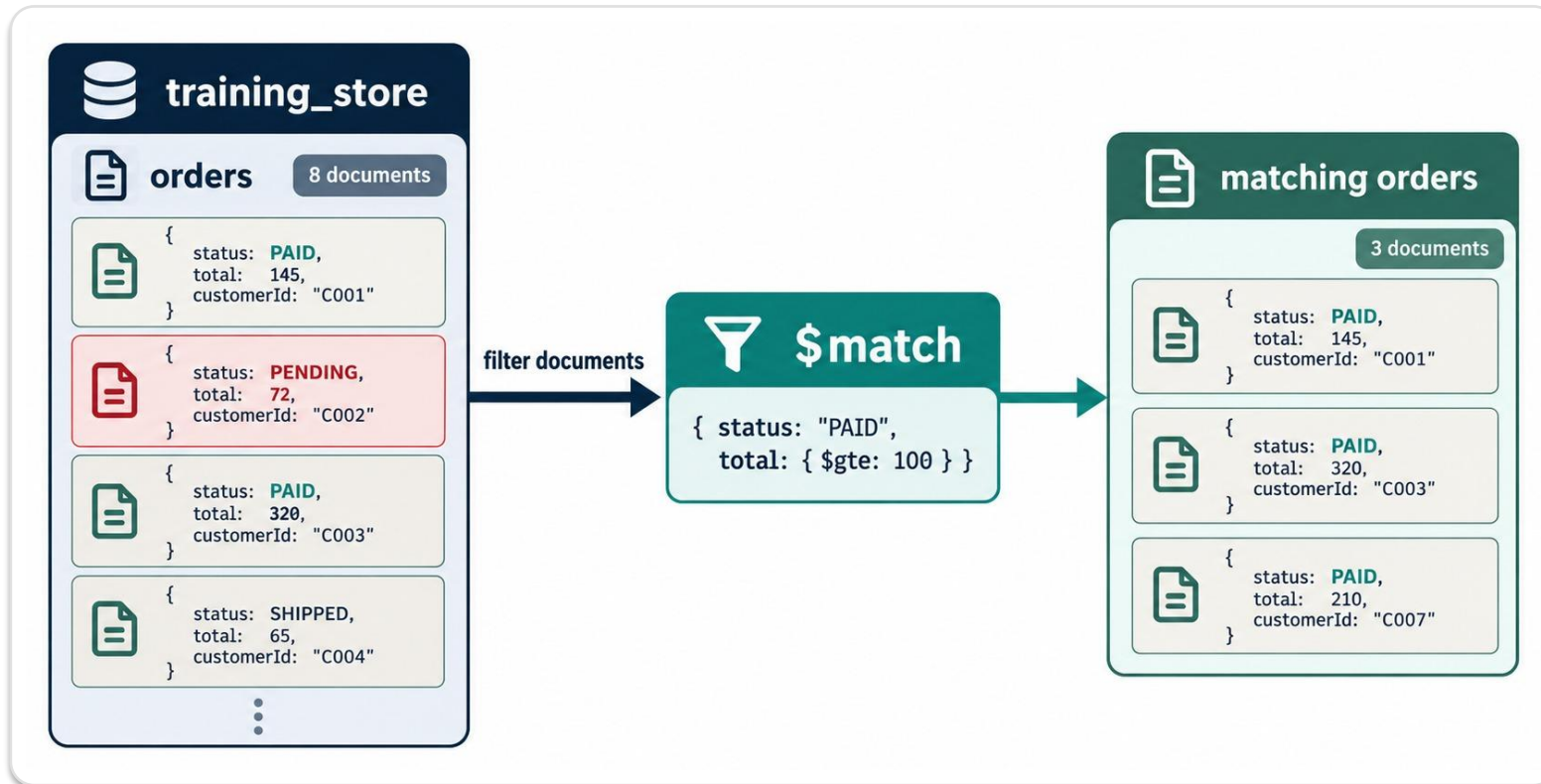
How do we keep only the documents we need, and give them the shape the report needs?

IN THIS PART

- 1 \$match: filter the stream
- 2 \$match on real fields and dates
- 3 \$project: include, exclude, rename
- 4 Calculated fields and \$set
- 5 \$sort and \$limit: top-N

THE \$MATCH STAGE

Keep only the documents that satisfy the filter.



Paid orders of 1000 or more

```
{ $match: {
  paymentStatus: "PAID",
  total: { $gte:
    Decimal128("1000.00") }
} }
```

Result

4 orders: O6101, O6202, O6301 and O6304.

Put it first

Later stages then handle 4 documents, not 17.

SAME LANGUAGE Every Module 4 filter works here.

KEY TAKEAWAY \$match filters with the find() query language — and belongs as early as possible.

\$MATCH ON REAL TRAINING_STORE FIELDS

Categories, a date range and a nested array path.

Active laptops and accessories

```
db.products.aggregate([
  { $match: {
    active: true,
    category: { $in:
      ["LAPTOP", "ACCESSORY"] }
  } }
])
// 7 products (XBAD is one)
```

Reviews written in August

```
db.reviews.aggregate([
  { $match: { createdAt: {
    $gte: ISODate("2026-08-01"),
    $lt:  ISODate("2026-09-01")
  } } }
])
// 3 reviews: L110, B300, P1001
```

Customers in Ontario or Quebec

```
db.customers.aggregate([
  { $match: {
    "addresses.province":
      { $in: ["Ontario", "Quebec"] }
  } }
])
// 3 customers: C101, C412, C515
```

Same operators

\$in, ranges and dotted paths behave exactly as in find().

Half-open dates

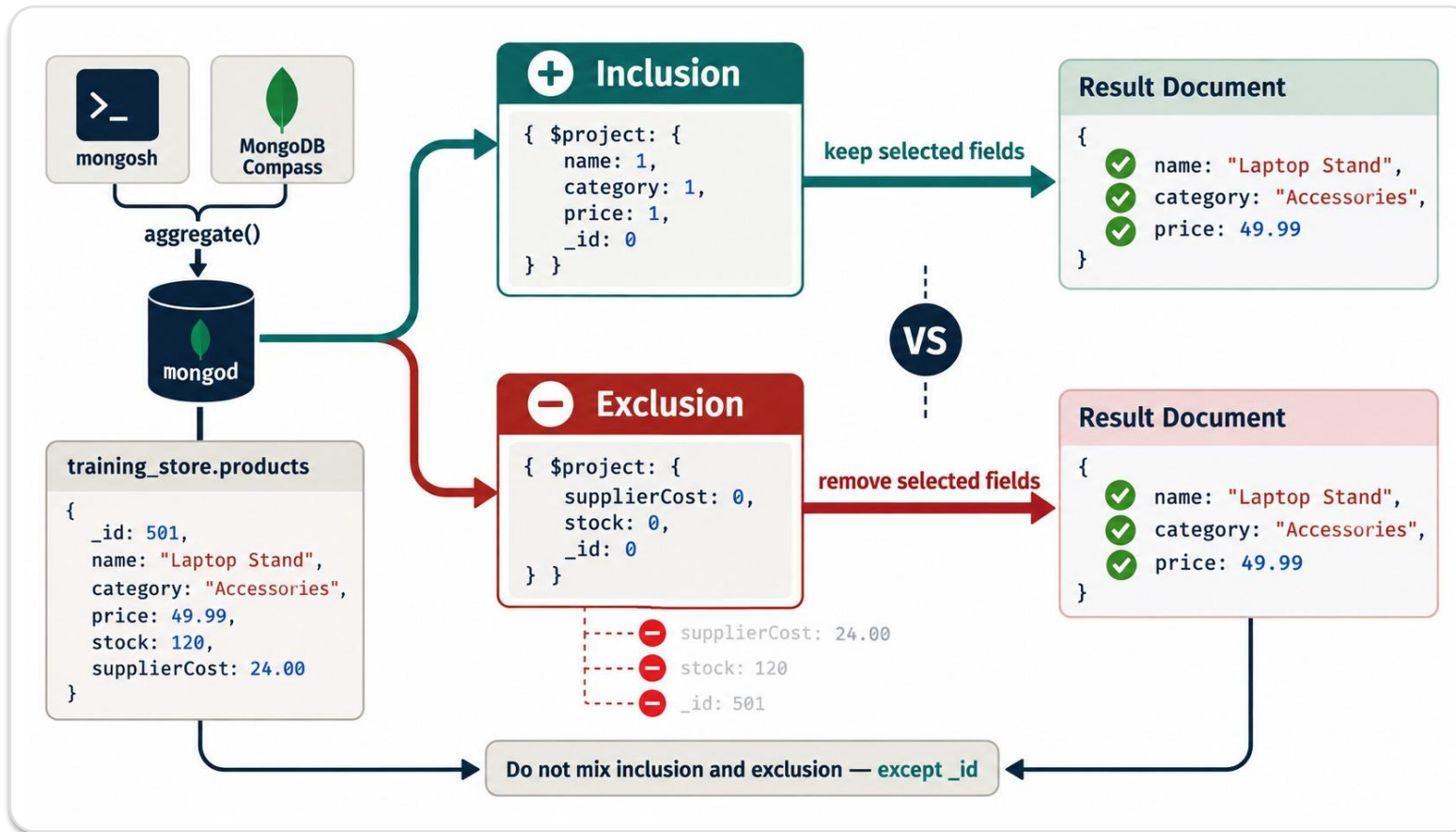
\$gte the first day, \$lt the next month — no gap, no overlap.

TRY IT Run each filter and check the count.

KEY TAKEAWAY \$match speaks the query language you already know — use the real field paths and types.

\$PROJECT: INCLUDE, EXCLUDE, RENAME

Decide exactly which fields leave the stage.



Rename sku, keep two fields

```
db.products.aggregate([
  { $project: { _id: 0,
    productCode: "$sku",
    name: 1, price: 1 } }
])
```

1 or 0

1 keeps a field, 0 drops it — never both, except `_id`.

Rename

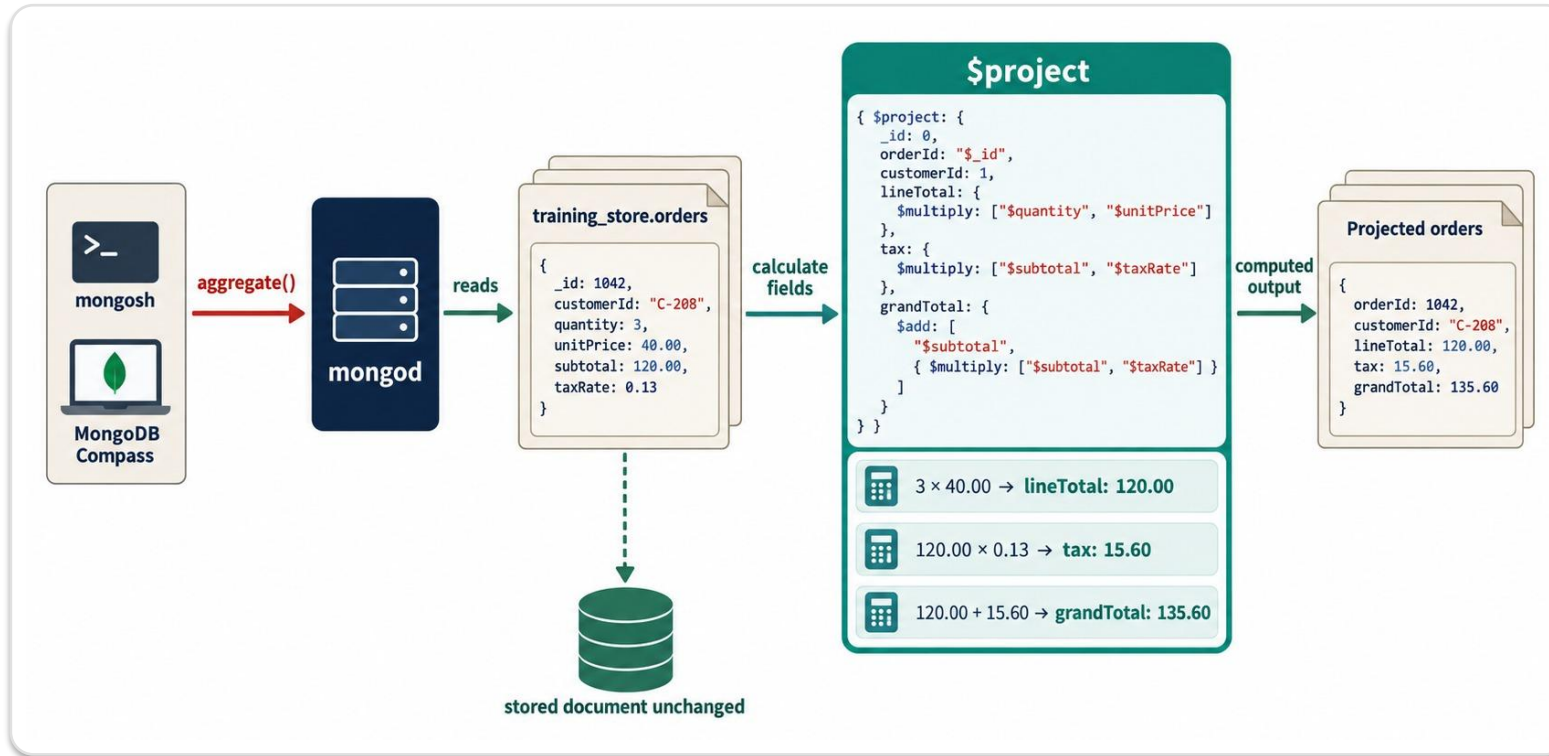
`productCode: "$sku"` copies the value under a new name.

OUTPUT ONLY Stored products are unchanged.

KEY TAKEAWAY `$project` decides the output shape: keep, drop, rename — the stored document stays the same.

CALCULATED FIELDS

Expressions compute values that aren't stored.



Line revenue (after \$unwind)

```
{ $set: { lineRevenue: {
  $multiply: ["$items.quantity",
    "$items.unitPrice"]
} } }
```

No lineTotal

training_store stores quantity and unitPrice — compute the line.

\$project or \$set

`$project` keeps only listed fields; `$set` adds fields and keeps the rest.

KEY TAKEAWAY Compute what the report needs with expressions — `$set` for helper fields, `$project` for the final shape.

CALCULATED FIELDS YOU CAN CHECK

Rebuild each order's total — and survive a missing shippingFee.

Pipeline

```
db.orders.aggregate([
  { $match: { orderNumber:
    { $in: ["06201", "06301"] } } },
  { $set: { shipping: { $ifNull:
    ["$shippingFee", Decimal128("0.00")] } } },
  { $set: { checkTotal: { $add:
    ["$subtotal", "$tax", "$shipping"] } } },
  { $project: { _id: 0, orderNumber: 1,
    total: 1, shipping: 1, checkTotal: 1 } }
])
```

✓ Result

```
{ orderNumber: "06201",
  total: Decimal128("79.07"),
  shipping: Decimal128("0.00"),
  checkTotal: Decimal128("79.07") }
{ orderNumber: "06301",
  total: Decimal128("1512.23"),
  shipping: Decimal128("15.00"),
  checkTotal: Decimal128("1512.23") }
```

\$ifNull

O6201 has no shippingFee, so 0.00 is used instead of null.

\$add

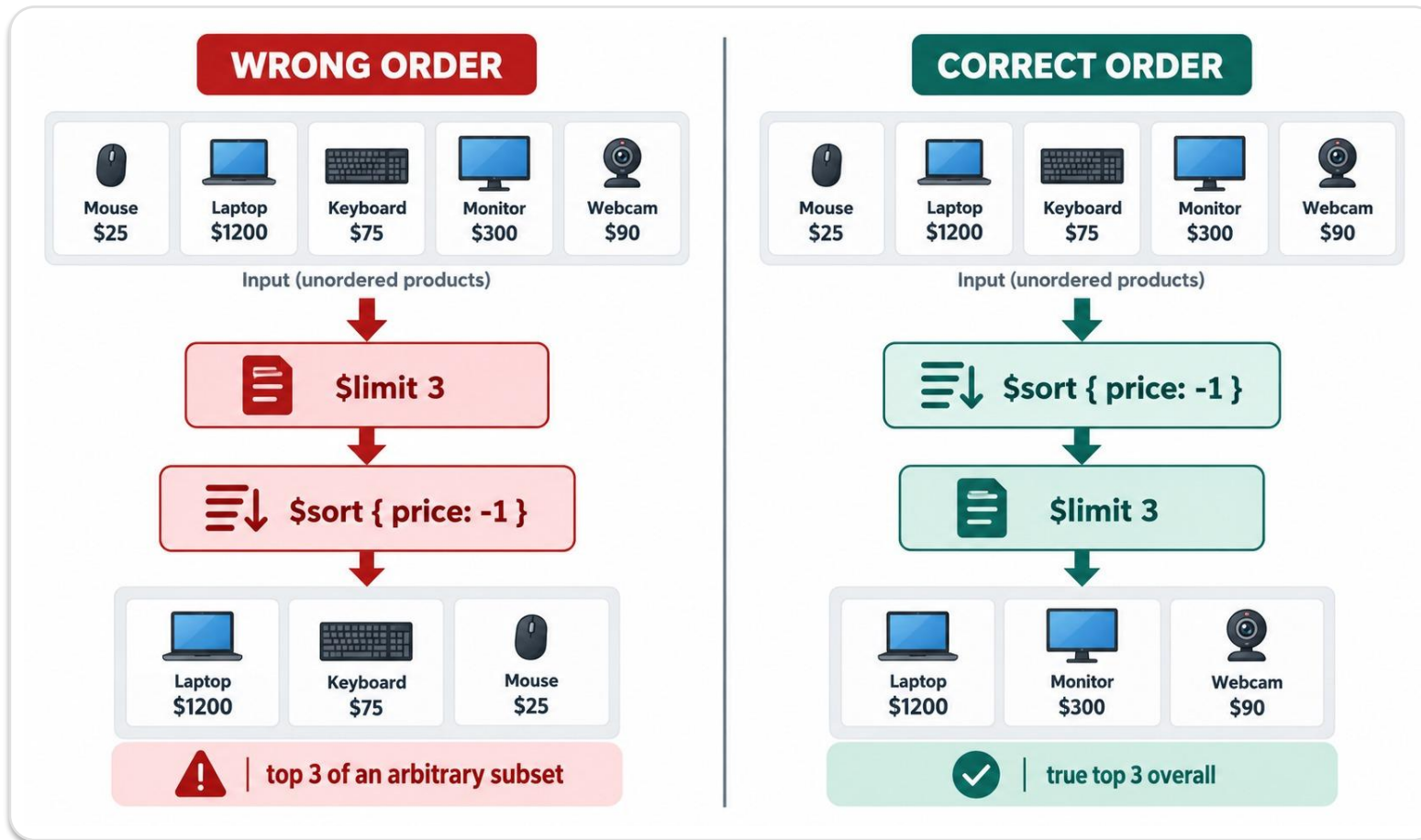
subtotal + tax + shipping rebuilds the stored total.

WHY CHECK checkTotal ≠ total means bad data — or a bad pipeline.

KEY TAKEAWAY Handle missing fields with \$ifNull before calculating — one null turns the whole result into null.

\$SORT BEFORE \$LIMIT

A top-N report sorts the whole stream, then keeps N.



Top three paid orders

```
{ $match: {  
  paymentStatus: "PAID" } },  
{ $sort: { total: -1,  
           orderNumber: 1 } },  
{ $limit: 3 }
```

Result

O6202 and O6304 tie at 2146.99; O6301 is third at 1512.23.

Tie-breaker

orderNumber makes the order repeatable.

\$skip

\$sort → \$skip → \$limit pages through results.

KEY TAKEAWAY Every top-N report is \$sort, then \$limit — with a tie-breaker when values can tie.

EXERCISE: BUILD A SHOE PRICE LIST

Filter, shape, sort and limit — then predict the output.

PRACTICE EXERCISE

10 MIN

SCENARIO

Marketing wants the two most expensive active shoes: name and price only, highest price first.

training_store has three shoes: S200, S210 and S290.

YOUR TASKS

- 1 Choose the four stages and their order
- 2 Write the \$match
- 3 Write the \$project
- 4 Add \$sort and \$limit; predict the output

EXPECTED OUTCOME

- ✓ `$match → $project → $sort → $limit`
- ✓ `{ active: true, category: "SHOE" }`
- ✓ `{ _id: 0, name: 1, price: 1 }`
- ✓ Running Shoe 129.99, then Trail Shoe 89.99

KEY TAKEAWAY Filter first, shape the output, then sort before you limit.

PART 3: GROUP AND SUMMARIZE

Many documents in, one summary per key out.



SO FAR

- ✓ \$match keeps the documents you need
- ✓ \$project and \$set shape documents
- ✓ Top-N is \$sort, then \$limit

NEXT QUESTION

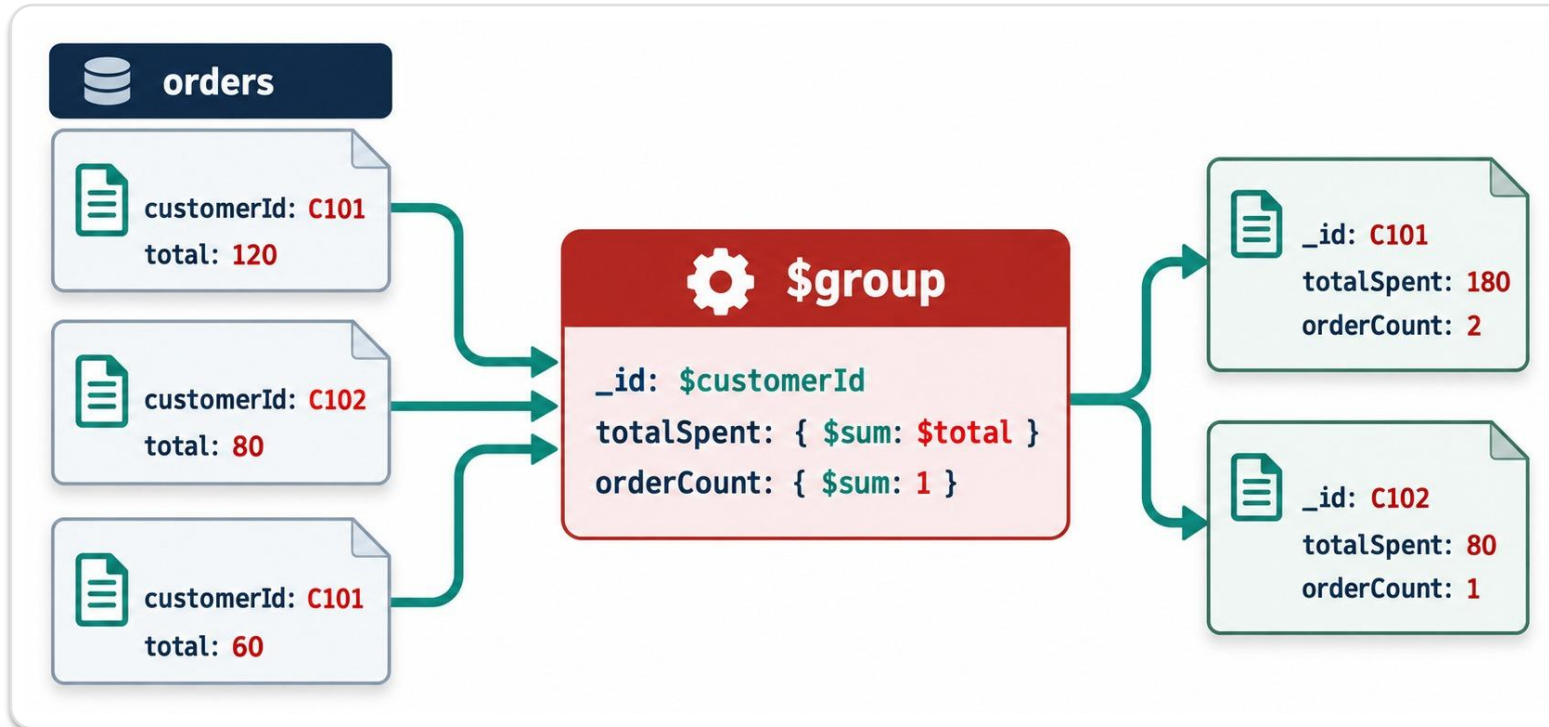
How do we turn many documents into one summary per customer, status or category?

IN THIS PART

- 1 \$group, _id and accumulators
- 2 Products by category
- 3 Grouping by several fields
- 4 What \$group keeps — and loses
- 5 \$count and \$sortByCount

\$GROUP: ONE DOCUMENT PER KEY

_id is the grouping key; every other field is an accumulator.



_id

"\$customerId" — one output per distinct value; null means one group.

Accumulators

\$sum, \$avg, \$min, \$max, \$first, \$last, \$push, \$addToSet.

Count or total

\$sum: 1 counts documents; \$sum: "\$total" adds values.

OUR DATA Aisha's 4 paid orders → spent 3418.62.

KEY TAKEAWAY \$group turns many documents into one per key — _id names the key, accumulators compute the rest.

PRODUCTS BY CATEGORY

Count, average, minimum and maximum price for active products.

Pipeline

```
db.products.aggregate([
  { $match: { active: true } },
  { $group: {
    _id: "$category",
    productCount: { $sum: 1 },
    averagePrice: { $avg: "$price" },
    minimumPrice: { $min: "$price" },
    maximumPrice: { $max: "$price" }
  } },
  { $sort: { averagePrice: -1 } }
])
```

✓ Result (rounded)

_id	count	averagePrice
LAPTOP	2	1599.99
SHOE	3	88.32
ACCESSORY	5	86.24
BOOK	2	49.99

L190 is inactive

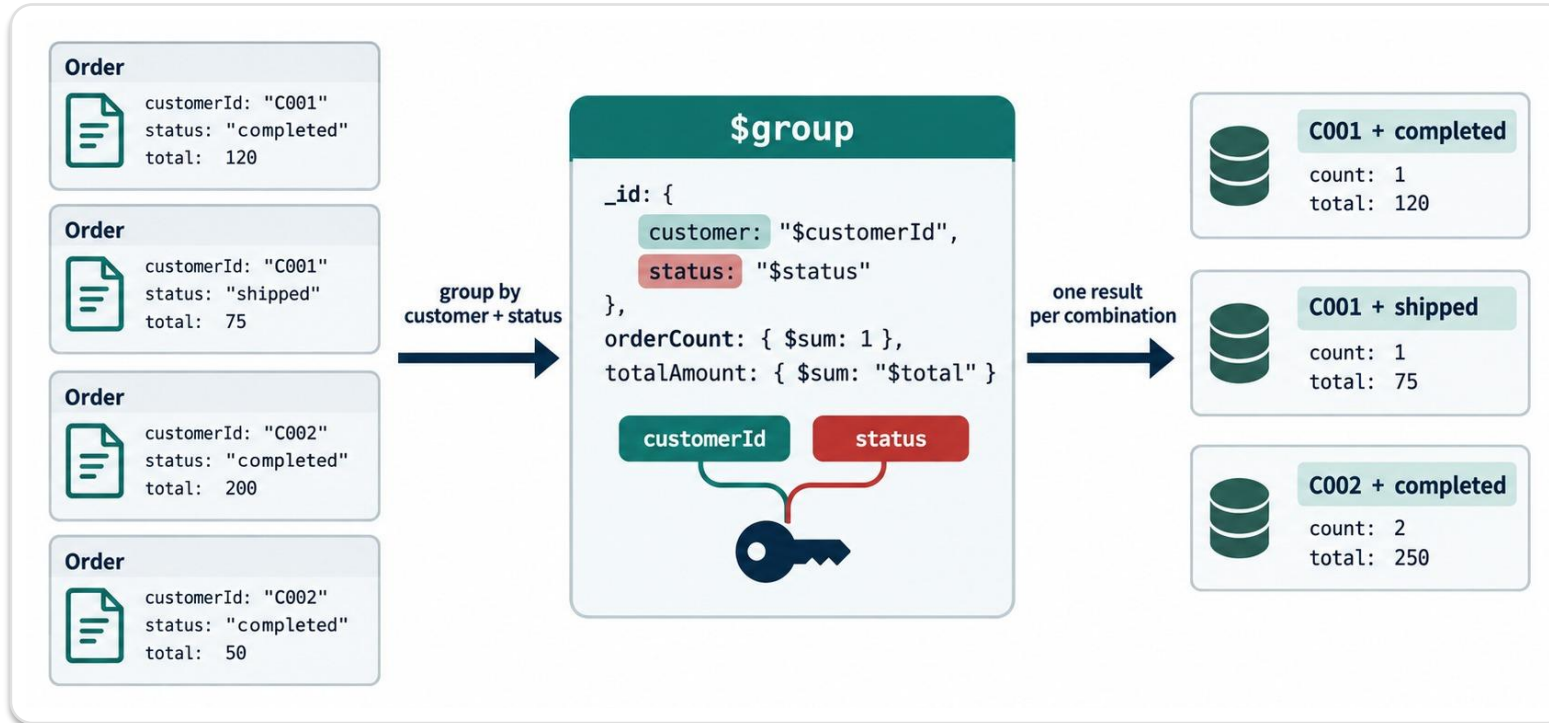
So LAPTOP counts 2, not 3.

XBAD TRAP Price "49.99" is a string: \$avg skips it, \$max returns it.

KEY TAKEAWAY Accumulators summarize each group — and a wrongly typed value changes the answer quietly.

GROUPING BY SEVERAL FIELDS

A document as `_id`: one group per combination of values.



Payment × fulfillment

```
{ $group: {  
  _id: {  
    pay: "$paymentStatus",  
    ful: "$fulfillmentStatus" },  
  orders: { $sum: 1 },  
  revenue: { $sum: "$total" } } }
```

Result

7 groups; PAID + SHIPPED is largest: 6 orders, 3809.48.

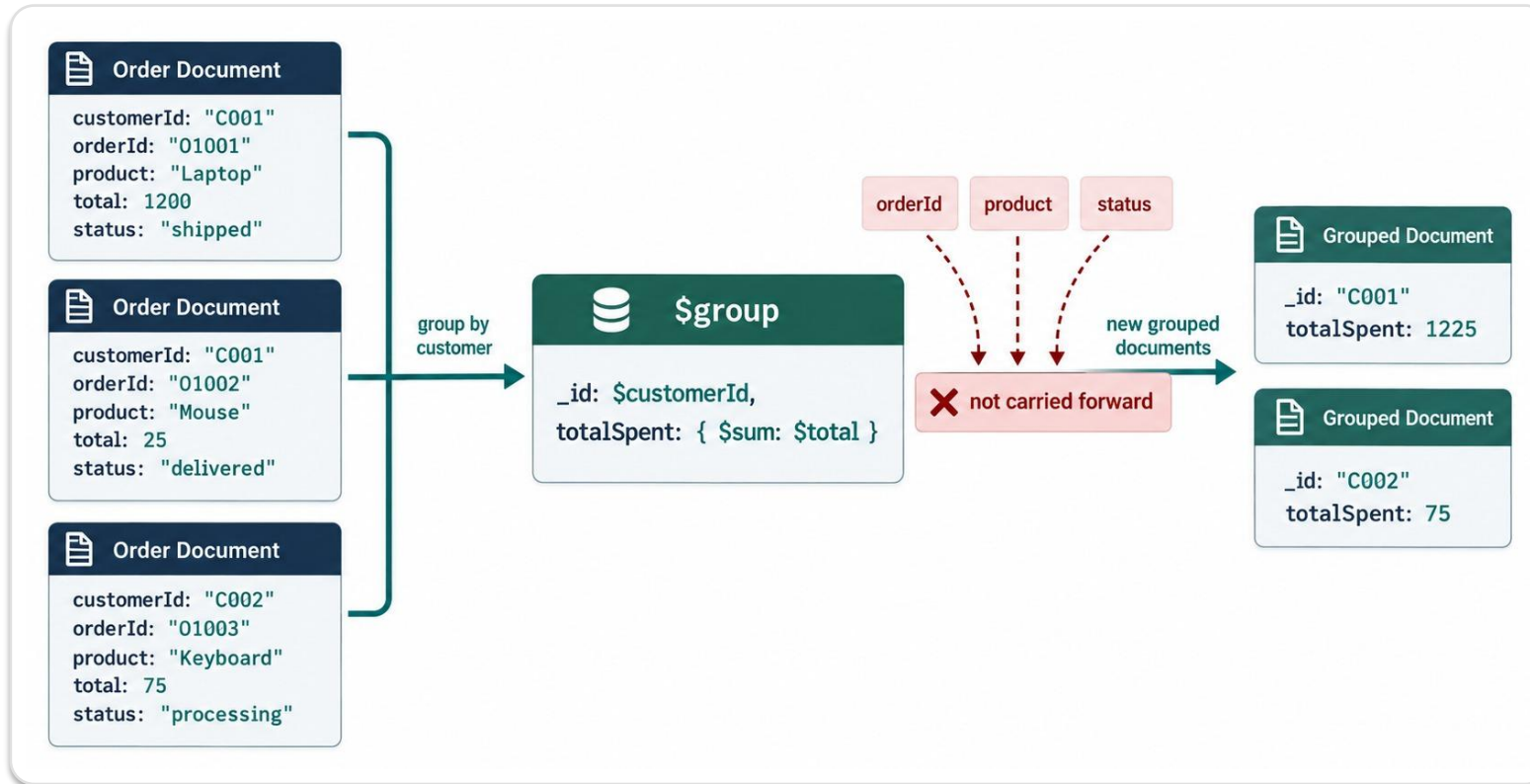
Read `_id` in words

{ PENDING, NEW } = all pending, new orders: O5001, O6499.

KEY TAKEAWAY A compound `_id` groups by every combination — read it as a sentence about the orders it holds.

WHAT \$GROUP KEEPS — AND LOSES

Only _id and the accumulators leave the stage.



Lost

orderId, items and fulfillmentStatus don't carry forward.

✓ Keep values with accumulators

```
{ $group: { _id: "$customerId",
  spent: { $sum: "$total" },
  orders: { $push: "$orderId" },
  statuses: { $addToSet:
    "$fulfillmentStatus" } } }
```

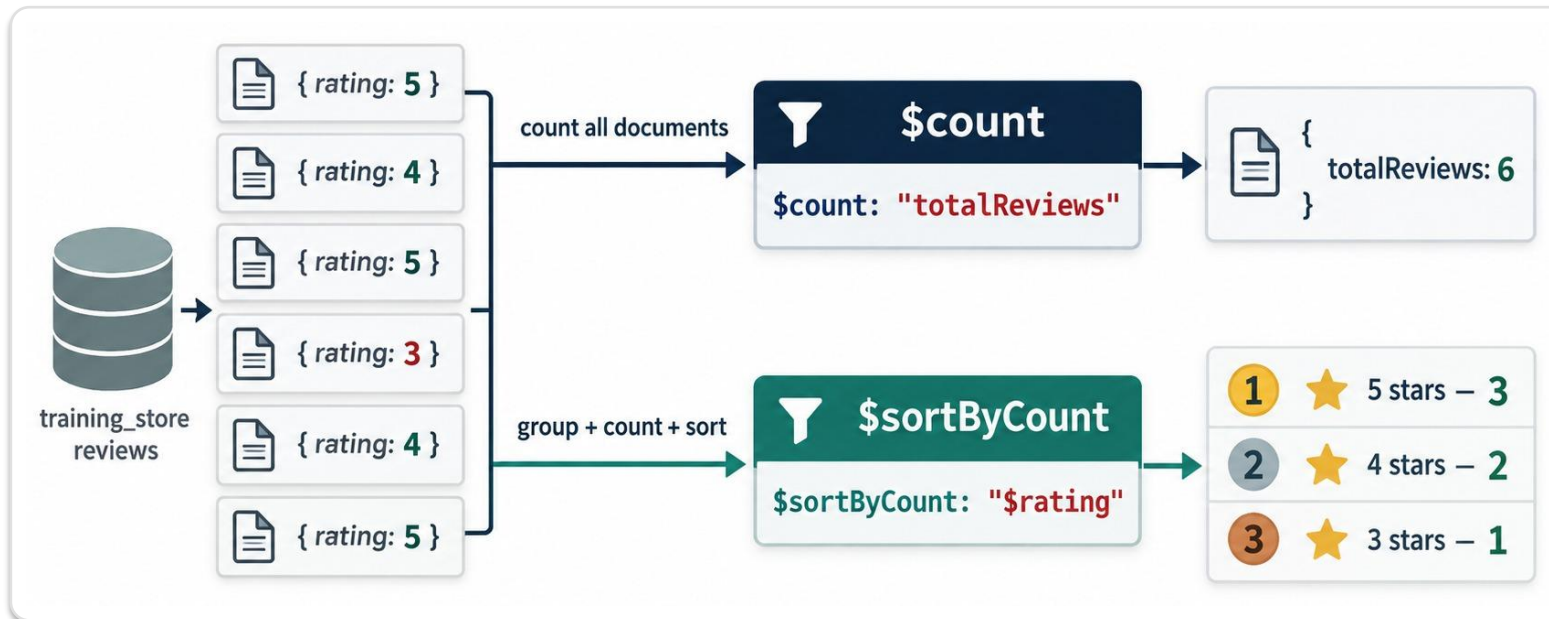
\$first and \$last

Sort first — then they pick a meaningful document.

KEY TAKEAWAY Anything a report needs after \$group must be kept by an accumulator.

\$COUNT AND \$SORTBYCOUNT

Count the stream — or count and rank each value.



Reviews per rating

```
db.reviews.aggregate([
  { $sortByCount: "$rating" }
])
```

Result

5 stars: 3 · 4 stars: 2 · 3 stars: 1 — our six reviews.

\$count

`{ $count: "totalReviews" } → { totalReviews: 6 }`

SHORTCUT `$sortByCount = $group + $sum: 1 + $sort.`

KEY TAKEAWAY `$count` counts what reaches it; `$sortByCount` counts and ranks each value in one stage.

EXERCISE: ORDERS BY PAYMENT STATUS

Group all orders — then check the counts add up.

PRACTICE EXERCISE

10 MIN

SCENARIO

Finance asks how many orders — and how much money — sit in each payment status.

Use all 17 orders, not only the paid ones.

YOUR TASKS

- 1 Group by paymentStatus
- 2 Count with \$sum: 1; add total with \$sum
- 3 Sort by revenue, highest first
- 4 Rename _id to paymentStatus in \$project

EXPECTED OUTCOME

- ✓ PAID: 13 orders, 9118.44
- ✓ PENDING: 3 orders, 4488.39
- ✓ FAILED: 1 order, 45.19
- ✓ $13 + 3 + 1 = 17$ orders

KEY TAKEAWAY Check a grouped report by adding its counts back to the input count.

PART 4: ARRAYS AND JOINS

Look inside the items array — and reach into other collections.



SO FAR

- ✓ \$group summarizes many documents per key
- ✓ Accumulators keep what the report needs
- ✓ Counts should add back up to the input

NEXT QUESTION

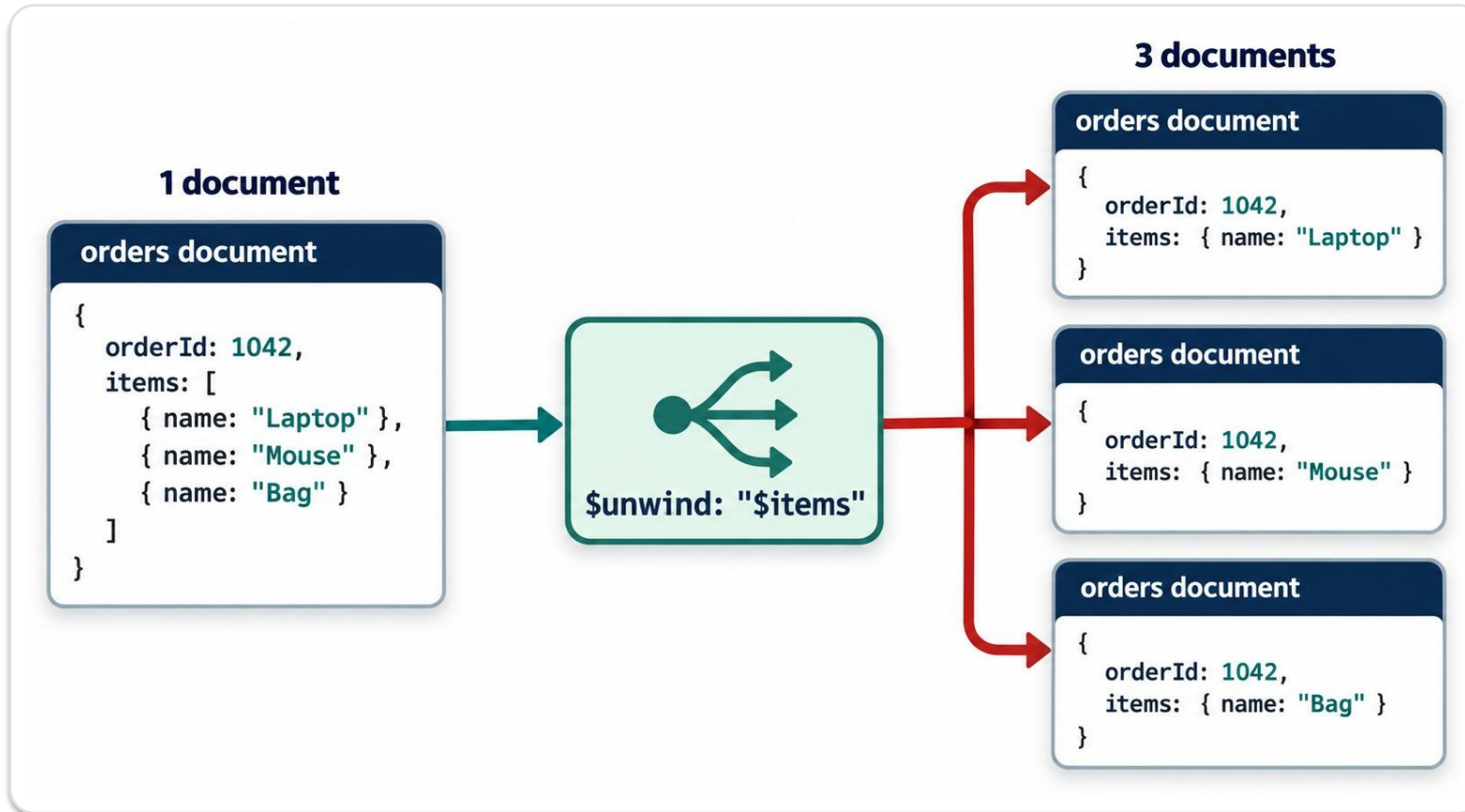
How do we report on individual order lines, and add customer names that live in another collection?

IN THIS PART

- 1 \$unwind: one document per item
- 2 Product sales from order lines
- 3 The double-counting trap
- 4 \$lookup: matched and unmatched
- 5 Customer spending with names

\$UNWIND: ONE DOCUMENT PER ARRAY ELEMENT

An order with three items becomes three documents.



Our three-item order

```
db.orders.aggregate([
  { $match: { orderNumber: "06303" } },
  { $unwind: "$items" }
])
```

Result

3 documents — items is now one object, not an array.

Scale

The 13 paid orders become 21 line documents.

OUTPUT ONLY Stored orders keep their arrays.

KEY TAKEAWAY \$unwind turns each array element into its own document — filter first so it multiplies less.

PRODUCT SALES FROM ORDER LINES

Paid orders → lines → line revenue → one row per SKU.

Pipeline (Exercises 5.1 and 5.4)

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $unwind: "$items" },
  { $set: { lineRevenue: { $multiply:
    ["$items.quantity", "$items.unitPrice"] } } },
  { $group: {
    _id: "$items.sku",
    unitsSold: { $sum: "$items.quantity" },
    revenue: { $sum: "$lineRevenue" },
    orderCount: { $sum: 1 } } },
  { $sort: { revenue: -1 } },
  { $limit: 5 }
])
```

✓ Result

_id	units	revenue	orders
L110	2	3799.98	2
L100	2	2599.98	2
A500	2	499.98	2
S200	3	389.97	2
B300	3	179.97	2

orderCount

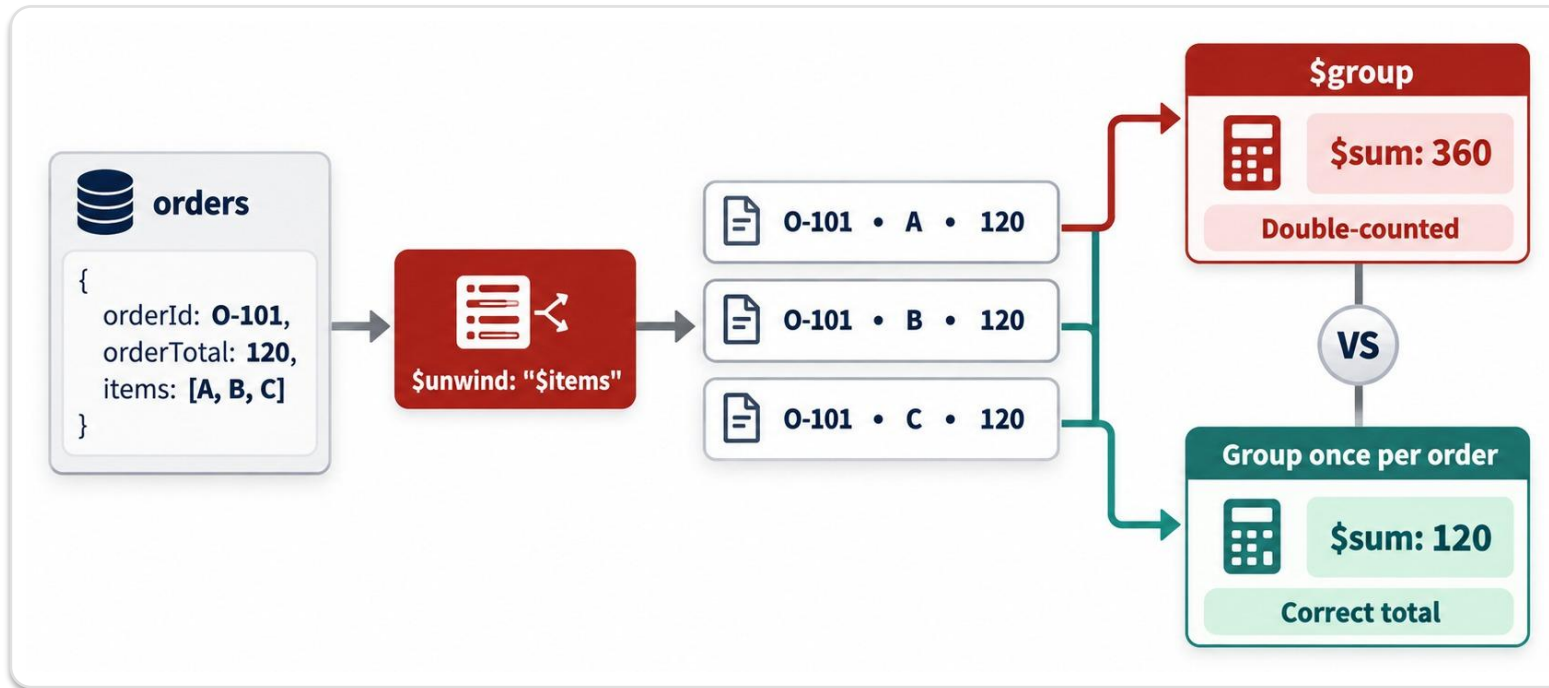
\$sum: 1 counts lines — equal to orders here, as no order repeats a SKU.

ORDER Match → unwind → compute → group.

KEY TAKEAWAY Unwind after filtering, compute the line value, then group — that is the product-report pattern.

DOUBLE-COUNTING AFTER \$UNWIND

Each line still carries the whole order's total.



The trap

O6303's total, 174.47, is summed three times — once per line.

On our data

Paid revenue is 9118.44; unwind first and it becomes 12119.25.

The rule

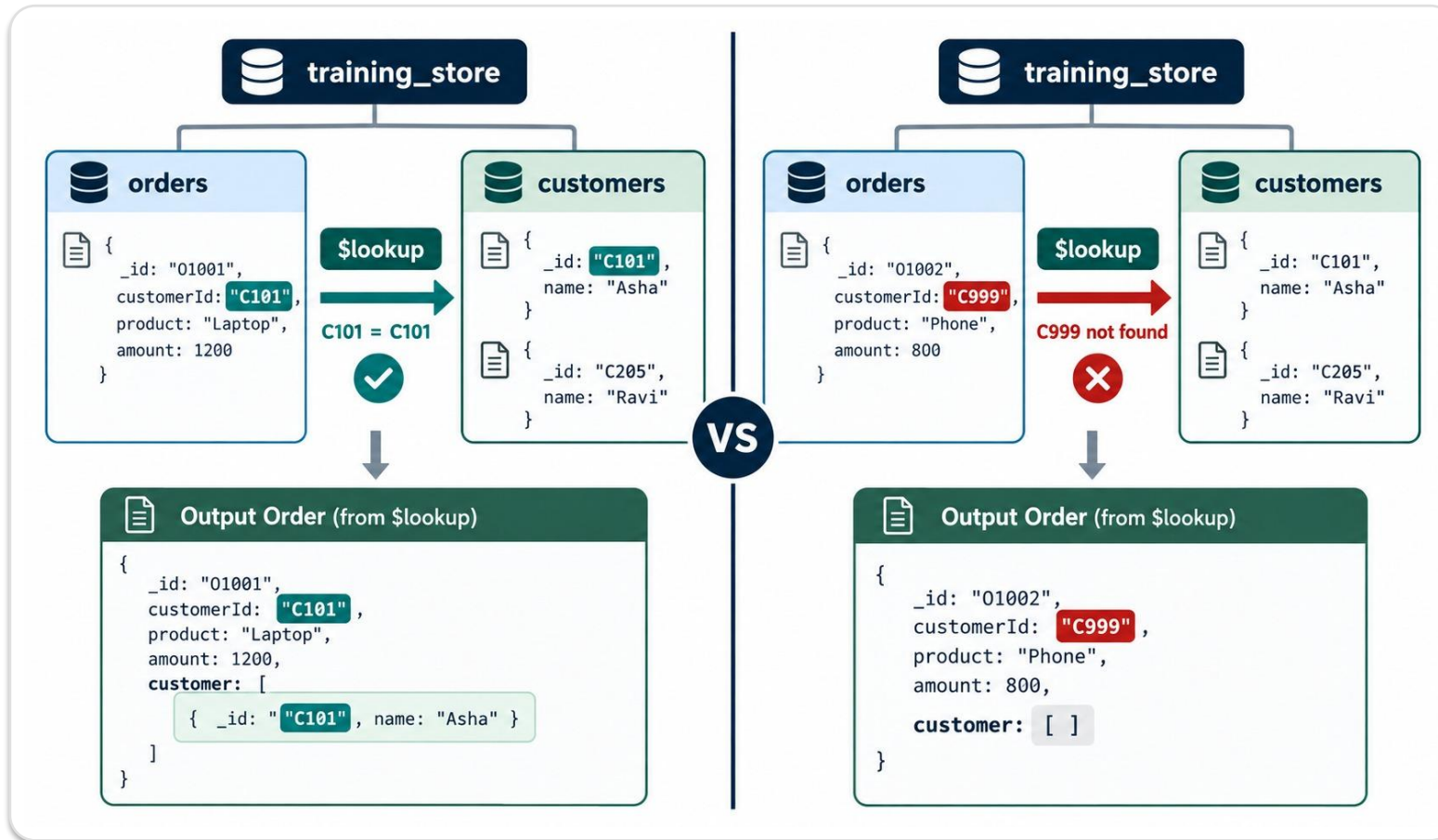
Sum order fields before \$unwind; sum line values after it.

CHECK Known totals must not change.

KEY TAKEAWAY After \$unwind, sum line values — never the order total that every line repeats.

\$LOOKUP: MATCHED AND UNMATCHED

Join another collection; the matches arrive as an array.



Orders → customers

```
{ $lookup: {
  from: "customers",
  localField: "customerId",
  foreignField: "_id",
  as: "customer" } }
```

Always an array

One match → [{...}]; no match → [].

O6499

Its customerId matches no one, so customer is [].

KEEP IT \$unwind with
preserveNullAndEmptyArrays: true.

KEY TAKEAWAY \$lookup is a left outer join — it never drops an order, it just leaves the array empty.

CUSTOMER SPENDING WITH NAMES

Group first, then join — five lookups instead of thirteen.

Pipeline

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $group: { _id: "$customerId",
    orders: { $sum: 1 },
    spent: { $sum: "$total" } } },
  { $lookup: { from: "customers",
    localField: "_id", foreignField: "_id",
    as: "customer" } },
  { $unwind: "$customer" },
  { $project: { _id: 0, orders: 1, spent: 1,
    name: { $concat: ["$customer.name.first",
      " ", "$customer.name.last"] } } },
  { $sort: { spent: -1 } }
])
```

✓ Result

name	orders	spent
Aisha Khan	4	3418.62
Maya Chen	2	2497.97
Jordan Lee	2	2321.46
Luis Romero	3	668.04
Priya Patel	2	212.35

No Sam Okonkwo

C515's only order, O6402, failed payment.

JOIN LATE Look up after \$group — one per customer.

KEY TAKEAWAY Group before you join — the lookup then runs once per customer, not once per order.

EXERCISE: HOW BIG DOES THE STREAM GET?

Predict the document count after each stage.

PRACTICE EXERCISE

5 MIN

SCENARIO

Before building the product report, predict how many documents each stage passes on.

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $unwind: "$items" },
  { $count: "lines" }
])
```

YOUR TASKS

- 1 Predict the count after `$match`
- 2 Predict the count after `$unwind`
- 3 Name the paid order with the most lines
- 4 What if `$unwind` came before `$match`?

EXPECTED OUTCOME

- ✓ 13 paid orders
- ✓ { lines: 21 }
- ✓ O6303 — three items
- ✓ 26 lines, filtered back to 21: more work

KEY TAKEAWAY Know how many documents each stage emits — it is the first check of any pipeline.

PART 5: CONDITIONS, DATES AND REPORTS

Per-document decisions, calendar reports and several metrics at once.



SO FAR

- ✓ \$unwind opens the items array
- ✓ Sum line values, never repeated totals
- ✓ \$lookup joins as an array — group first

NEXT QUESTION

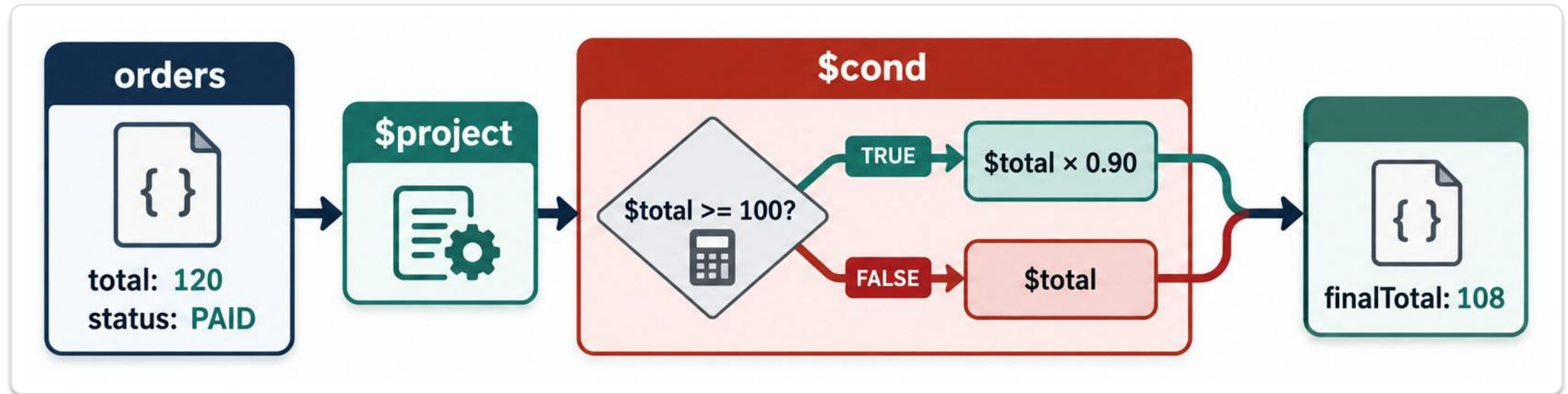
How do we label, bucket and date-group documents — and return a whole dashboard from one call?

IN THIS PART

- 1 \$cond and \$switch
- 2 Dates and time zones
- 3 Monthly paid sales
- 4 \$bucket price bands
- 5 \$facet: several reports at once
- 6 A paid-orders dashboard

\$COND: DECIDE PER DOCUMENT

If the condition is true use one value, otherwise the other.



Inside \$set

```
orderSize: { $cond: [
  { $gte: ["$total",
    Decimal128("1000.00")] },
  "HIGH_VALUE", "STANDARD" ] }
```

Result

O6201 (79.07) → STANDARD; O6202 (2146.99) → HIGH_VALUE. Paid orders: 4 HIGH_VALUE, 9 STANDARD.

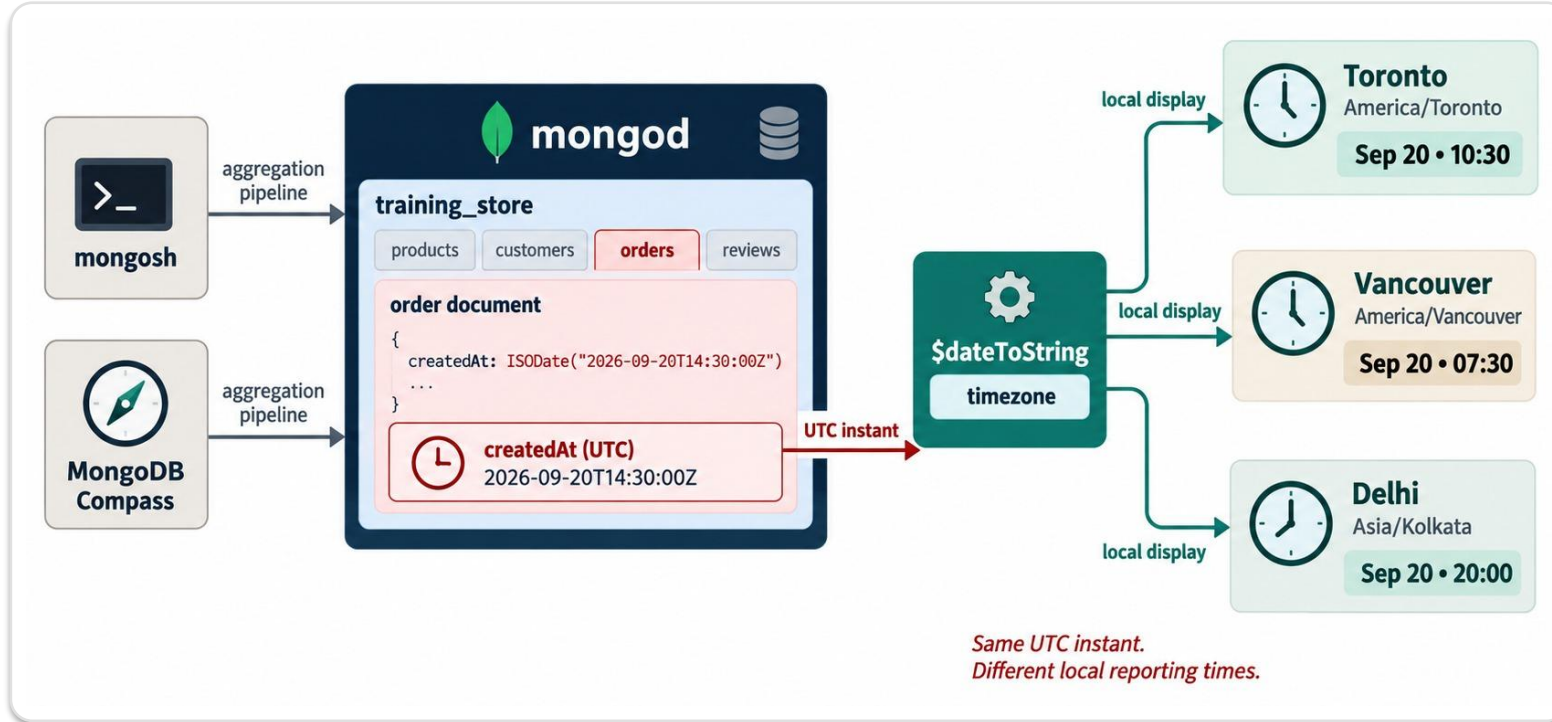
\$switch

Several cases in order; the first true case wins, and default catches the rest.

KEY TAKEAWAY `$cond` and `$switch` put business rules into the pipeline — one decision per document.

DATES AND TIME ZONES

Dates are stored in UTC; reports need a local calendar.



A month key

```
month: { $dateToString: {  
  format: "%Y-%m",  
  date: "$createdAt",  
  timezone: "America/Toronto"  
} }
```

Our order

O5001's createdAt, 14:30 UTC, is 10:30 in Toronto.

Name the zone

Day and month boundaries depend on it.

ALSO \$year, \$month, \$dateTrunc, \$dateAdd.

KEY TAKEAWAY Store dates in UTC; state the time zone every time a report groups by day or month.

MONTHLY PAID SALES

A month key, then one row per month in calendar order.

Pipeline

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $set: { month: { $dateToString: {
    format: "%Y-%m", date: "$createdAt",
    timezone: "America/Toronto" } } } },
  { $group: { _id: "$month",
    orders: { $sum: 1 },
    revenue: { $sum: "$total" } } },
  { $sort: { _id: 1 } }
])
```

✓ Result

_id	orders	revenue
2026-07	3	1835.95
2026-08	4	2731.92
2026-09	6	4550.57

Sorts as text

"YYYY-MM" strings sort in calendar order.

O5001 excluded

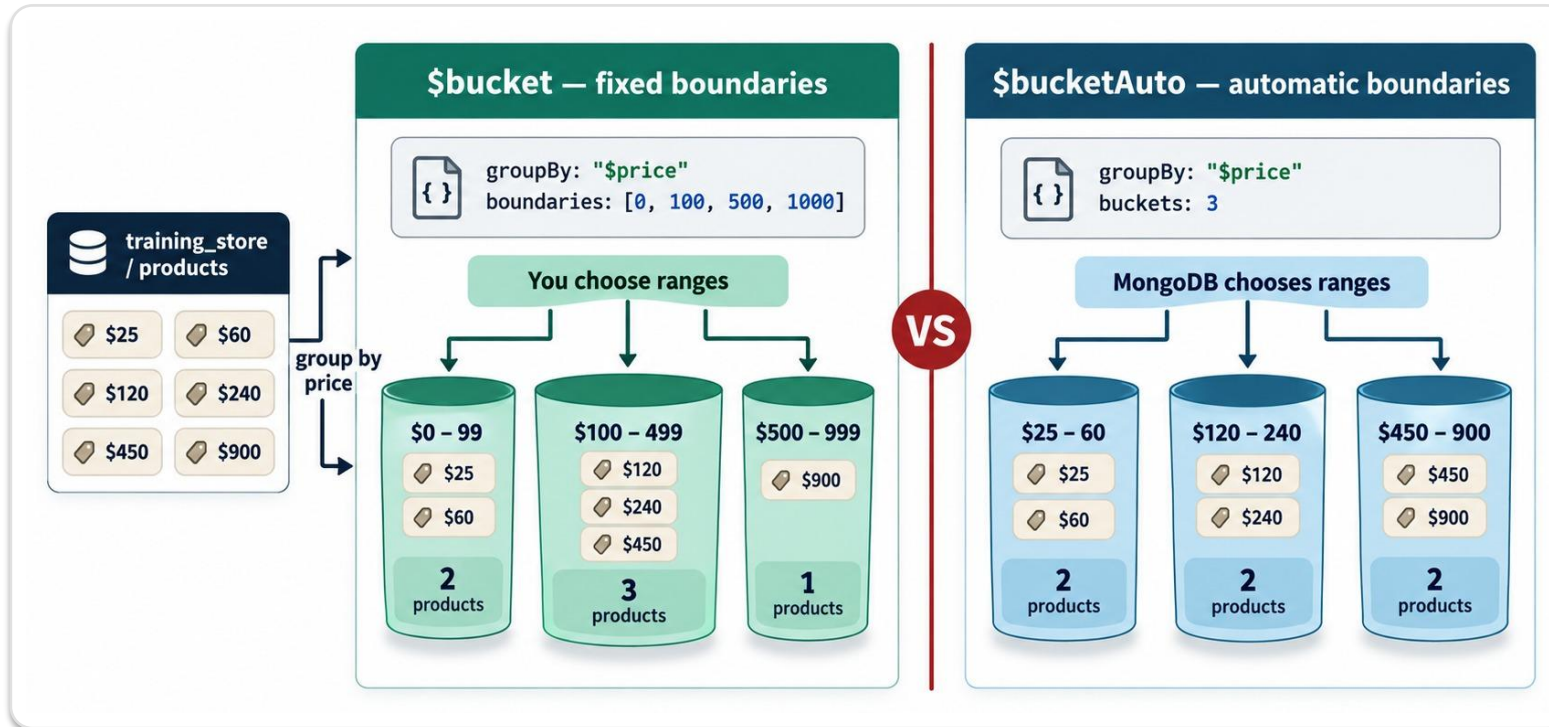
Its payment is still PENDING.

CHECK 3 + 4 + 6 = 13 paid orders.

KEY TAKEAWAY A date expression builds the key, \$group builds the rows, \$sort puts them in calendar order.

\$BUCKET: PRICE BANDS

Group values into ranges the business defines.



Price bands on products

```
{ $bucket: {
  groupBy: "$price",
  boundaries: [0, 50, 100, 500, 2000],
  default: "Other",
  output: { count: { $sum: 1 } } } }
```

Result

0: 5 · 50: 2 · 100: 3 · 500: 2 · Other: 1

Boundaries

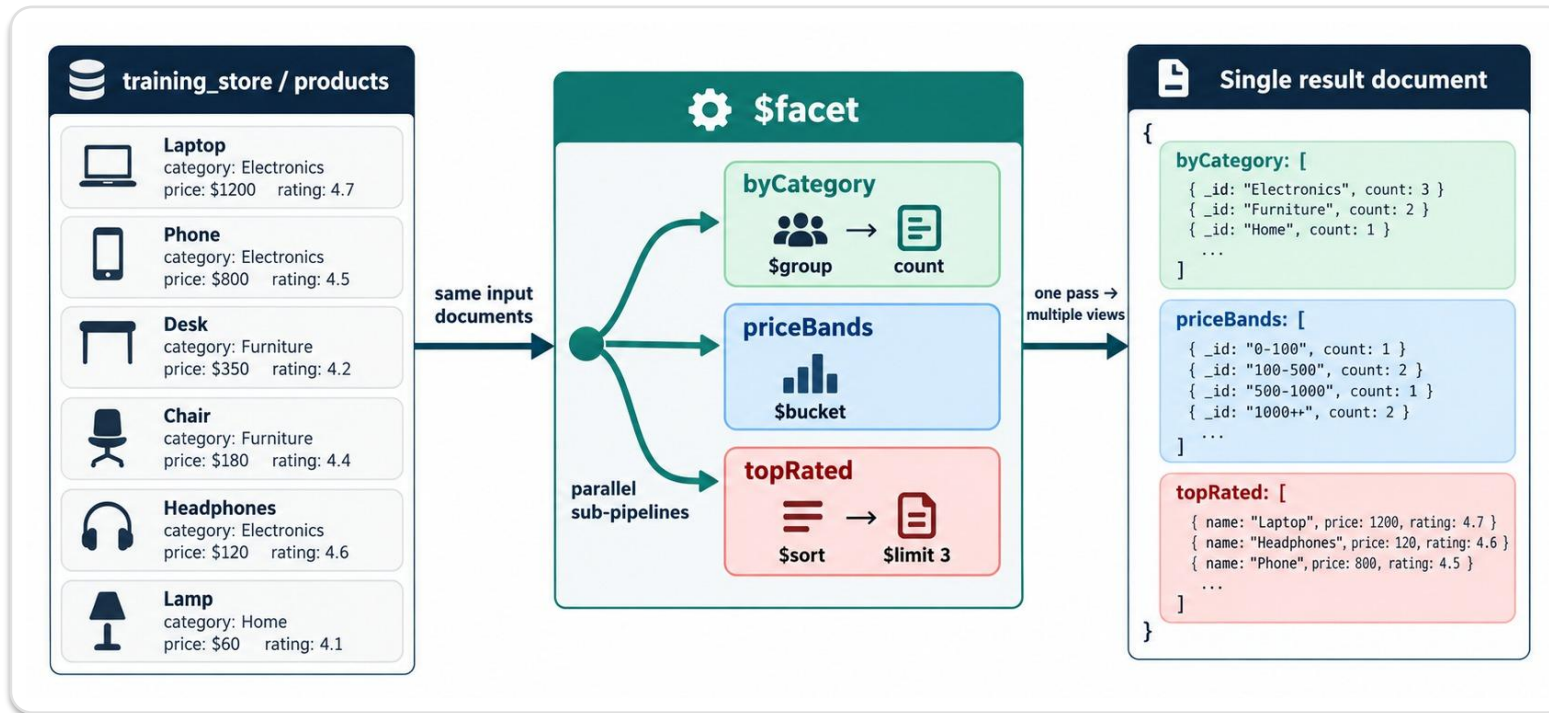
Lower bound in, upper bound out: 49.99 → the 0 band.

OTHER = XBAD A string price fits no band.

KEY TAKEAWAY \$bucket turns a number into a business band — and default catches whatever doesn't fit.

\$FACET: SEVERAL REPORTS IN ONE PASS

Parallel sub-pipelines on the same input, one result document.



Same input

Every sub-pipeline sees the documents that reach \$facet.

One document

Each facet name holds an array of results.

Use it for

Dashboards and API responses with several metrics.

FILTER FIRST One \$match before \$facet serves every facet.

KEY TAKEAWAY \$facet returns a whole dashboard in one document — one array per sub-pipeline.

A PAID-ORDERS DASHBOARD

Totals, a status breakdown and a high-value count in one call.

Pipeline (trimmed)

```
db.orders.aggregate([
  { $match: { paymentStatus: "PAID" } },
  { $facet: {
    totals: [ { $group: { _id: null,
      orders: { $sum: 1 },
      revenue: { $sum: "$total" } } } ],
    byFulfillment: [
      { $sortByCount: "$fulfillmentStatus" } ],
    highValue: [
      { $match: { total:
        { $gte: Decimal128("1000.00") } } },
      { $count: "orders" } ]
  } }
])
```

✓ Result: one document

```
{ totals: [ { _id: null,
  orders: 13, revenue: 9118.44 } ],
  byFulfillment: [
    { _id: "SHIPPED", count: 6 },
    { _id: "DELIVERED", count: 4 },
    { _id: "PROCESSING", count: 2 },
    { _id: "NEW", count: 1 } ],
  highValue: [ { orders: 4 } ] }
```

ARRAYS Even one-row facets are arrays.

KEY TAKEAWAY Filter once, then let each facet answer its own question — the result is one document.

PART 6: PERFORMANCE AND DEBUGGING

Right first, then cheap — and a method for finding what went wrong.



SO FAR

- ✓ \$cond, dates and \$bucket label documents
- ✓ \$facet returns several reports at once
- ✓ Every report was checked against the data

NEXT QUESTION

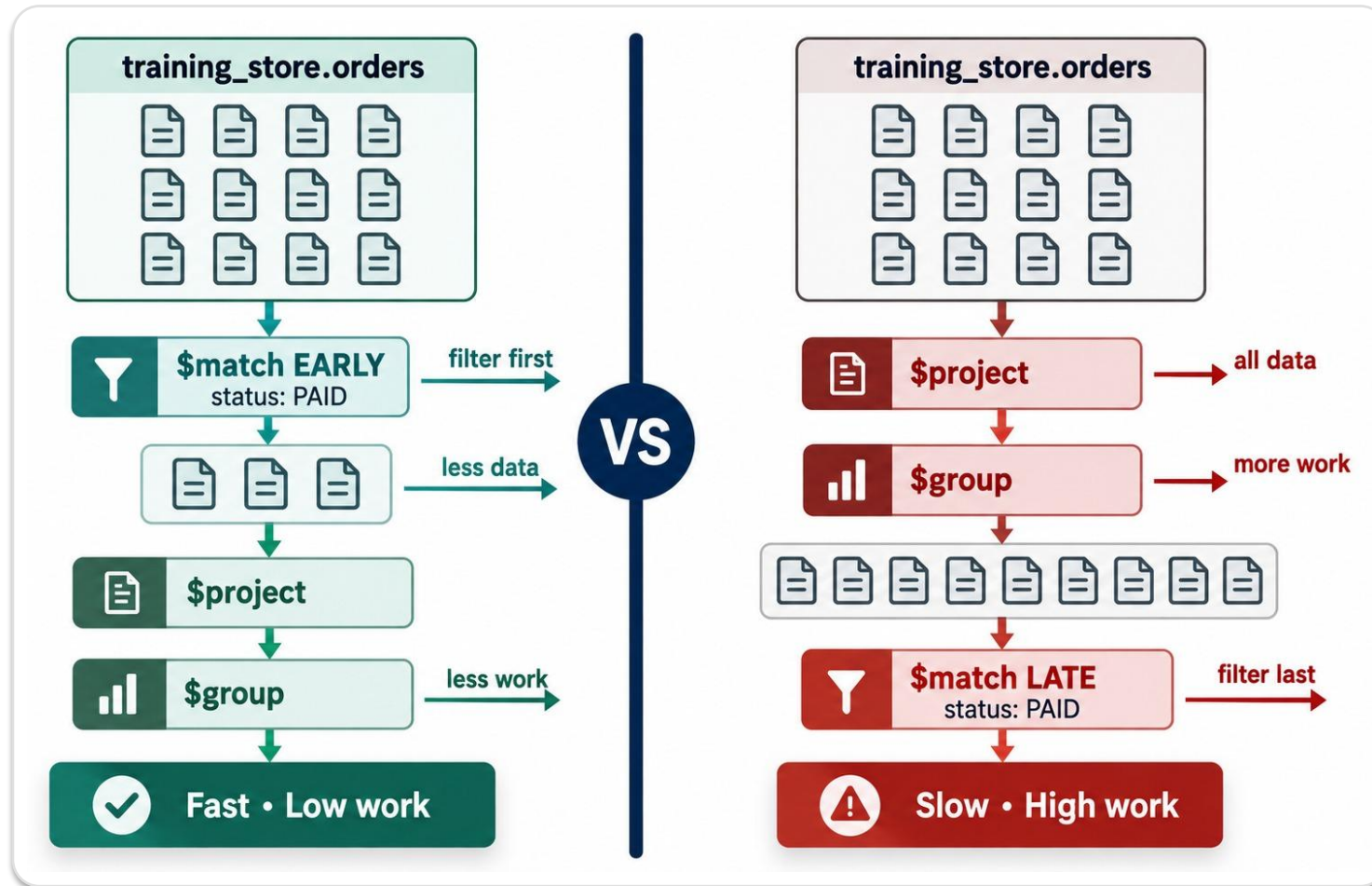
How do we order stages so a pipeline stays correct and cheap — and find the stage that broke it?

IN THIS PART

- 1 Early versus late \$match
- 2 Build and verify one stage at a time
- 3 Common aggregation mistakes

EARLY VS. LATE \$MATCH

Filter first: fewer documents for every later stage.



Early

\$match PAID first: 13 documents flow on, not 17.

Late can be wrong

After \$group, the status field no longer exists.

Join late

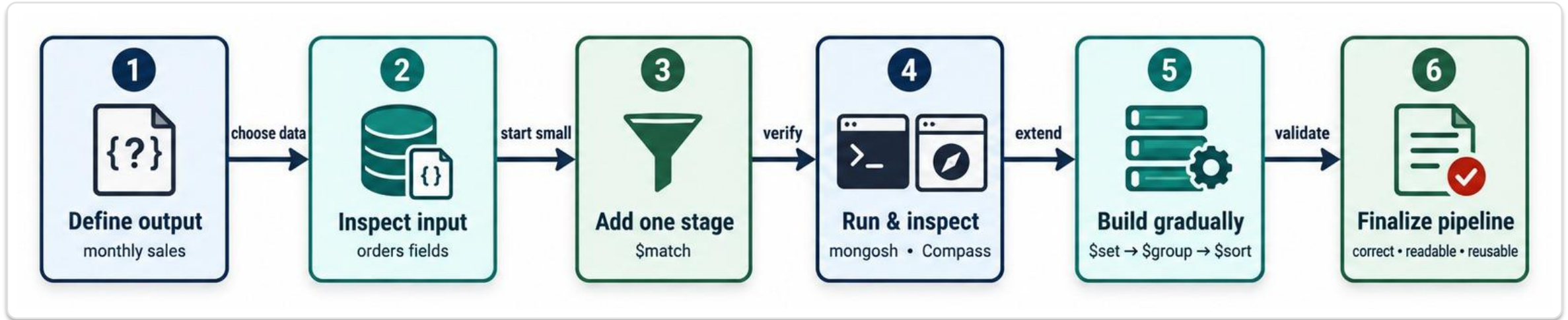
\$lookup after \$group runs once per customer.

INDEX A first-stage \$match can use one — Module 6.

KEY TAKEAWAY Filter as early as possible, and join after you have reduced the stream.

BUILD A PIPELINE ONE STAGE AT A TIME

Define the output, then add and check one stage at a time.



One stage at a time

Run the first stage and check the count and one document; add the next, run again.

Prove one number

Add Aisha's paid orders by hand: 3418.62 must match the pipeline.

Then inspect the plan

```
db.orders
  .explain("executionStats")
  .aggregate([ ... ])
```

KEY TAKEAWAY Build incrementally and verify each stage — a wrong path shows up the moment you add it.

COMMON AGGREGATION MISTAKES

Why a pipeline returns nothing, too much or the wrong total.

MISCONCEPTION

REALITY

"total" and "\$total" mean the same

Without \$ it is the text "total", not the field

\$limit, then \$sort gives a top N

Sort first — otherwise it is an arbitrary N

Sum total after \$unwind

Each order's total repeats on every line

\$match on paymentStatus after \$group

\$group dropped it — filter before grouping

\$lookup returns one document

It returns an array — unwind it or take [0]

ANTI-PATTERNS TO AVOID

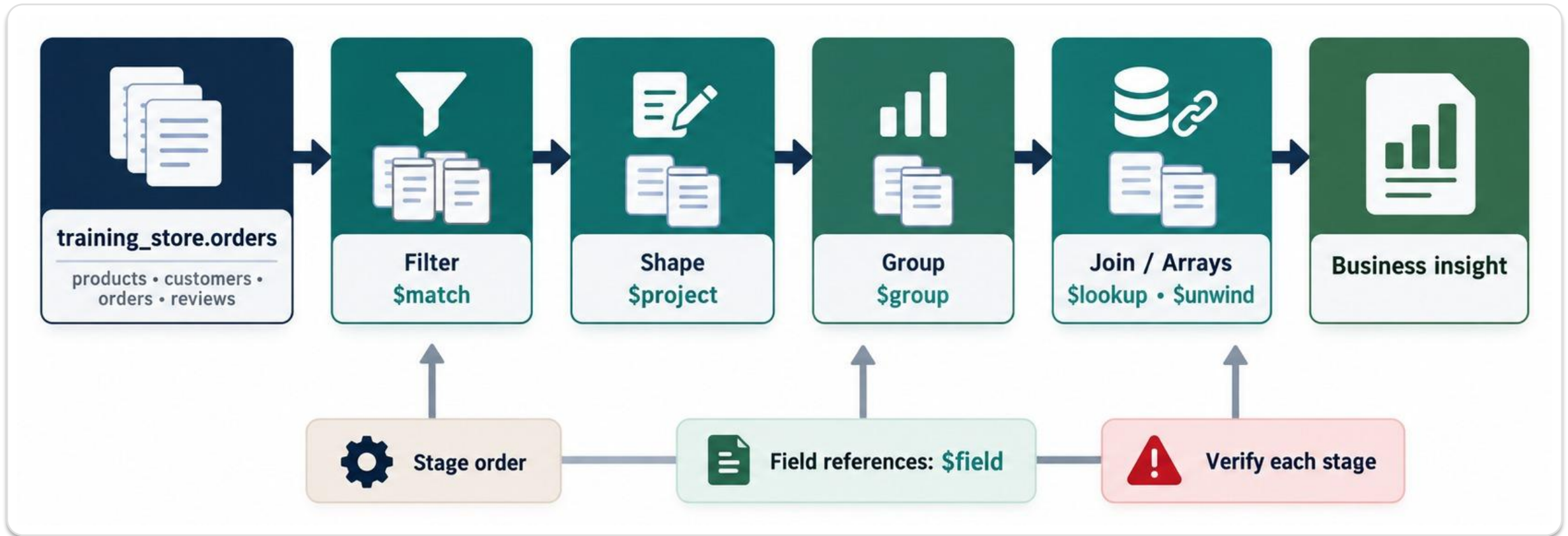
- ✗ status: "Shipped" — orders have no status field
- ✗ \$items.lineTotal — compute quantity × unitPrice
- ✗ \$lookup on every order before filtering
- ✗ Unwinding just to sum order totals
- ✗ Money compared with strings, not Decimal128

REMEMBER Filter early, group before joining, verify each stage.

KEY TAKEAWAY Most broken pipelines have a wrong field path or a stage in the wrong place.

MODULE 5 AT A GLANCE

Filter, shape, group, join — and verify each stage.



KEY TAKEAWAY A pipeline is ordered transformations: filter, shape, group and join — then check the numbers.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What does each pipeline stage receive and pass on?

2 Why can \$group then \$match on paymentStatus return nothing?

3 How is \$project different from \$set?

4 What does _id mean in a \$group stage?

5 What does \$sum: 1 calculate?

6 When do you need \$unwind?

7 What does \$lookup put in its as field?

8 Why must \$sort come before \$limit?

9 How do you handle a missing shippingFee?

10 What does a \$facet stage return?

EXERCISE 5.1 — ARRANGE PIPELINE STAGES

Module 5 checkpoint A: put six stages in the right order.

OFFICIAL CHECKPOINT A

10 MIN

labs/day-02/exercises/exercise-5.1-arrange-pipeline-stages.md

SCENARIO

Top five products by line revenue from paid orders. Use each stage once. No MongoDB needed.

\$limit	\$unwind	\$sort
\$match	\$group	\$set

YOUR TASKS

- 1 Propose an order for the six stages
- 2 Describe the documents after each one
- 3 Compare with the instructor's reveal
- 4 Explain why \$limit can't precede \$sort

SOLUTION (AFTER DEBRIEF)

- ✓ \$match → \$unwind → \$set
- ✓ → \$group → \$sort → \$limit
- ✓ \$unwind before grouping by SKU
- ✓ \$match after \$group filters groups

KEY TAKEAWAY The right order follows the data: filter, expand, compute, group, rank, cut.

EXERCISE 5.2 — BUILD A TOP-N REPORT

Module 5 checkpoint B: four rankings, each sorted before it is limited.

OFFICIAL CHECKPOINT B

15 MIN

labs/day-02/exercises/exercise-5.2-build-a-top-n-report.md

SCENARIO

Build four top-N reports from training_store.

- 1 Top 5 customers by paid spend
- 2 Top 3 paid orders by total
- 3 Top 3 SKUs by units sold
- 4 Products per category
{ \$sortByCount: "\$category" }

YOUR TASKS

- 1 Match PAID; group spend by customerId
- 2 Sort total -1 with a tie-breaker
- 3 Unwind items; sum quantity by SKU
- 4 Rank categories with \$sortByCount

SOLUTION (AFTER DEBRIEF)

- ✓ C101 first: 3418.62
- ✓ O6202, O6304 (2146.99), O6301
- ✓ A410 leads with 4 units
- ✓ ACCESSORY 5, LAPTOP 3, SHOE 3, BOOK 2

KEY TAKEAWAY Every top-N is \$sort then \$limit — add a tie-breaker whenever values can tie.

EXERCISE 5.3 — JOIN RELATED COLLECTIONS

Module 5 checkpoint C: add customer names, keep the unmatched order.

OFFICIAL CHECKPOINT C

20 MIN

labs/day-02/exercises/exercise-5.3-join-related-collections.md

SCENARIO

List every order with its customer number and full name, largest total first.

```
{ $lookup: { from: "customers",
  localField: "customerId",
  foreignField: "_id",
  as: "customer" } },
{ $unwind: { path: "$customer",
  preserveNullAndEmptyArrays:
    true } },
{ $project: ... }, { $sort: ... }
```

YOUR TASKS

- 1 Join orders to customers
- 2 Unwind, keeping empty matches
- 3 Project number and \$concat name
- 4 Find the order with no customer

SOLUTION (AFTER DEBRIEF)

- ✓ 17 rows; O6401 first (2945.98)
- ✓ Names such as Aisha Khan
- ✓ O6499: no customerNumber
- ✓ Kept by preserveNullAndEmptyArrays

KEY TAKEAWAY \$lookup returns an array — decide deliberately what happens when it is empty.

EXERCISE 5.4 — CORRECT A BROKEN PIPELINE

Module 5 checkpoint D: find the defects, then rewrite and run it.

OFFICIAL CHECKPOINT D

15 MIN

labs/day-02/exercises/exercise-5.4-correct-a-broken-pipeline.md

SCENARIO

This pipeline should report product revenue.

```
{ $limit: 3 },
{ $group: { _id: "items.sku",
  unitsSold: { $sum: "$quantity" },
  revenue: { $sum: "$total" } } },
{ $sort: { revenue: -1 } },
{ $lookup: { from: "customers", ... } },
{ $project: { sku:
  "$customer.customerNumber" } }
```

YOUR TASKS

- 1 List at least five defects
- 2 Name the symptom each one causes
- 3 Rewrite: match, unwind, set, group, sort, limit
- 4 Run it and name the top SKU

SOLUTION (AFTER DEBRIEF)

- ✓ \$limit first; no PAID filter
- ✓ "items.sku" lacks \$: one group
- ✓ No \$unwind; quantity is under items
- ✓ total is not line revenue
- ✓ No \$lookup needed; L110 is first

KEY TAKEAWAY Debug a pipeline stage by stage: field paths first, then stage order.

LAB 4 — AGGREGATION PIPELINE

Day 2 hands-on: top product categories for completed paid sales.

OFFICIAL LAB

60 MIN

labs/day-02/lab4/LAB-4-GUIDE.md

SCENARIO

Rank categories for PAID orders that are SHIPPED or DELIVERED. Reload load.js first.

```
$match → $unwind → $set lineSales  
→ $lookup products → $unwind  
→ $group by category → $set  
→ $match ≥ 50 → $sort → $limit
```

YOUR TASKS

- 1 0 Reload; count the collections
- 2 1 Count the completed paid orders
- 3 2 Unwind; compute lineSales
- 4 3 Join products; group by category
- 5 4 Prove one number; 5 explain

EXPECTED RESULTS

- ✓ 13 · 6 · 17 · 6; 13 PAID
- ✓ 10 orders, 18 lines
- ✓ quantity × unitPrice, Decimal128
- ✓ LAPTOP 4499.97; 4 categories
- ✓ 3 laptop units; \$match first

KEY TAKEAWAY One pipeline, built and checked stage by stage, turns 17 orders into a ranked category report.

MODULE 5 SUMMARY AND WHAT'S NEXT

From one \$match to a multi-metric sales report.

YOU CAN NOW

- ✓ Explain how documents flow through a pipeline
- ✓ Filter and shape with \$match, \$project, \$set
- ✓ Summarize with \$group and accumulators
- ✓ Expand items with \$unwind; join with \$lookup
- ✓ Build monthly, banded and \$facet reports
- ✓ Order stages for correct, cheap pipelines

NEXT: MODULE 6 — INDEXING AND QUERY PERFORMANCE

- 1 Which index supports our first \$match?
- 2 How do I read explain("executionStats")?
- 3 When is a query covered by an index?

BRING ALONG Your Day 2 Lab 4 pipeline and the index you would add for its first \$match.

KEY TAKEAWAY Module 6 makes these pipelines fast: the first \$match is where an index pays off.